物联网系统设计

实验 7：与乐鑫语音助手对话

姓名: 消瘦的猪猪

日期: 2025-3-31

Contents

1 实验任务与要求 **3**

2 实验平台与器材 **3**

3 实验过程及结果 **3**

3.1 ESP-IDF 3

3.1.1 获取 API Key 和 Secret Key 3

3.1.2 修改示例程序 5

3.1.3 选择大模型 6

3.1.4 编译烧录 7

3.2 屏幕配置 9

3.3 演示视频 10

4 实验思考 **11**

5 实验心得体会 **11**

1 实验任务与要求

1. ESP-IDF
2. 大模型对话
3. 演示视频

2 实验平台与器材

ESP-IDF、ESP32 模组

3 实验过程及结果

3.1 ESP-IDF

3.1.1 获取 API Key 和 Secret Key

- 安装 ESP-IDF
- 下载示例程序
- 激活百度千帆大模型，点击系统管理与统计列表下的应用接入

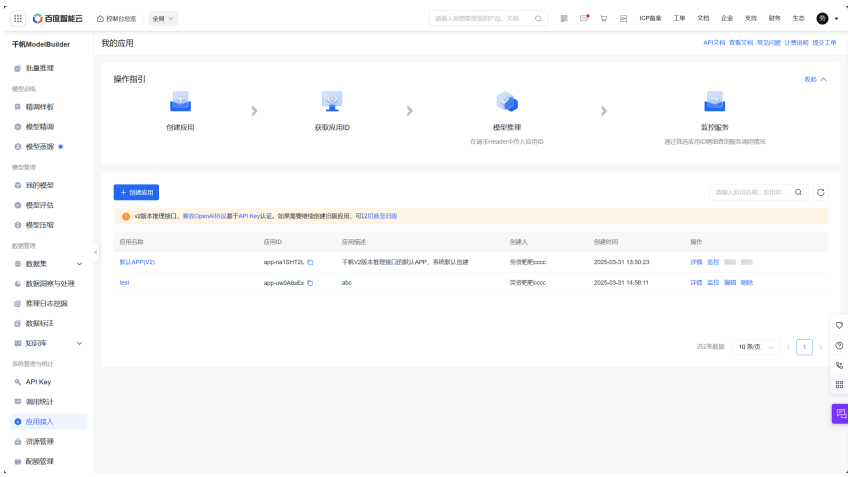


Figure 1: 应用接入

- 创建应用，自己输入应用名称和应用描述即可

Figure 2: 创建应用

- 保存应用的 API Key 和 Secret Key
- 激活百度智能云语音技术，点击应用列表

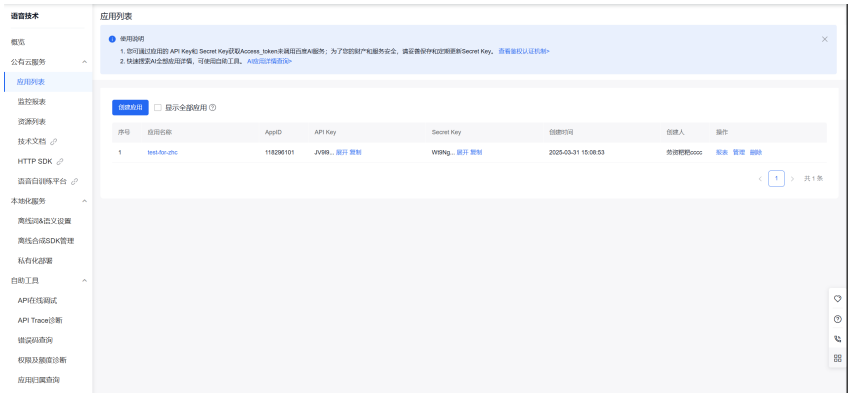


Figure 3: 应用列表

- 创建新应用，语音技术全选，应用归属选择个人

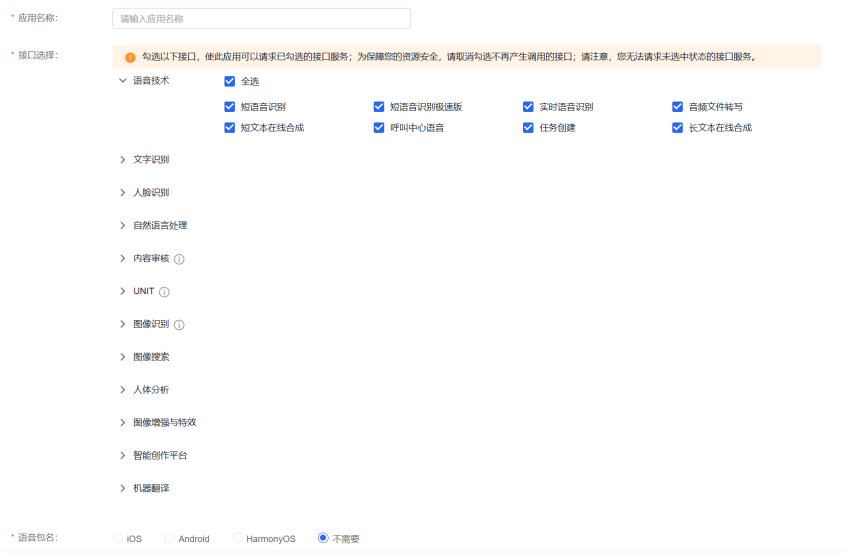


Figure 4: 创建新应用

- 保存语音技术的 API Key 和 Secret Key

3.1.2 修改示例程序

- 在 VScode 里打开文件夹
- 点击 **README** 文件，可以看到获取 **Access Token** 的方法，即将对应的 http 网址中的**你的 ID** 和**你的 secret** 分别换成百度 LLM 和百度 AUDIO 对应的 API Key 和 Secret Key，在浏览器中输入修改后的网址，获取 **access**$_{t}oken$

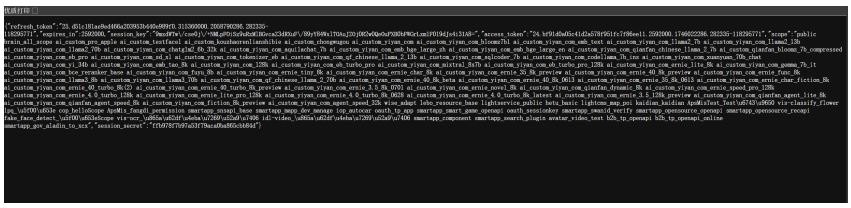


Figure 5: LLM:access token

Figure 6: Audio:access token

- 打开 **Kconfig.projbuild** 文件，将刚刚得到的 token 输入到对应位置，并修改 WIFI 的 ID 和密码，修改成自己手机热点的 ID 和密码

```
menu "Example Configuration"

    config BAIDU_AUDIO_ACCESS_TOKEN
        string "BAIDU AUDIO ACCESS TOKEN"
        default "24.4718de61176af22ef8cb4a118396ca8e.2592000.1746003774.282335-118296101"
        help
            ACCESS TOKEN for tts and stt

    config BAIDU_LLM_ACCESS_TOKEN
        string "BAIDU LLM ACCESS TOKEN"
        default "24.f142e9298f33ccaa50de020008f4077c.2592000.1746001606.282335-118295771"
        help
            ACCESS TOKEN for baidu LLM

    config EXAMPLE_WIFI_SSID
        string "SSID"
        default "babac"
        help
            SSID of the WiFi network to connect to

    config EXAMPLE_WIFI_PASSWORD
        string "Password"
        default "123123123"
        help
            Password of the WiFi network to connect to

endmenu
```

Figure 7: Kconfig

3.1.3 选择大模型

- 在百度智能云的模型广场，查找所需大语言模型，我分别使用了 ERNIE Lite 和 ERNIE 4.0 Turbo 两种大模型，两者功能对比见附件
- 打开对应大模型的 API 文档

Figure 8: 查看 API 文档

- 调用 API 示例如下

请求结构

以访问凭证access_token鉴权方式为例，说明调用API请求结构，示例如下。

Bash 复制

```bash
POST /rpc/2.0/ai_custom/v1/wenxinworkshop/chat/{创建服务时填写的API地址}?access_token=24.4a3a19b****
Host: aip.baidubce.com
Content-Type: application/json

{
  "messages": [
   {"role":"user","content":"介绍一下北京"}
  ]
}
```

Figure 9: API 请求

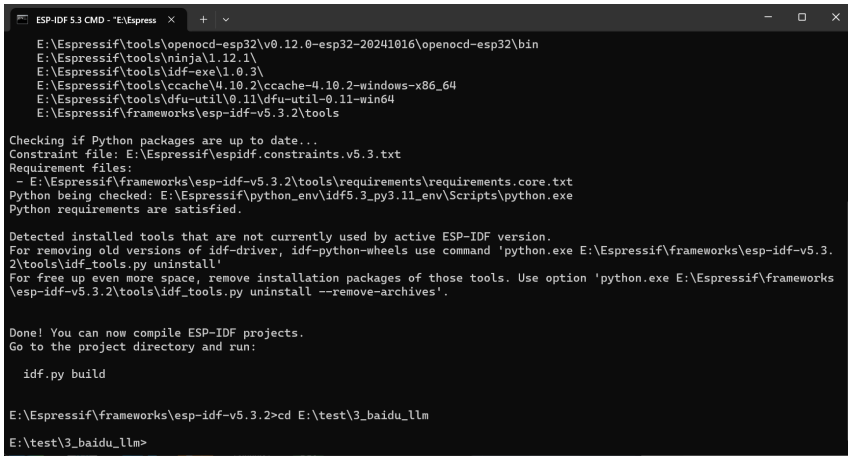
- 修改 baidu.c 文档中BAIDU_LLM_URL的宏定义

```c
    char *result_value = strdup(result_node->valuestring);

    cJSON_Delete(root);
    return result_value;

err:
    cJSON_Delete(root);
    return NULL;
}

#define BAIDUBCE_MESSAGE_FORMAT "{\"messages\":[{\"role\":\"user\",\"content\":\"%s\"}],\"system\":\"你的回答要尽量简洁，不超过 200 字\",\"stream\":false}"
#define BAIDU_LLM_URL           "https://aip.baidubce.com/rpc/2.0/ai_custom/v1/wenxinworkshop/chat/ernie-lite-8k?access_token=%s"
//#define BAIDU_LLM_URL           "https://aip.baidubce.com/rpc/2.0/ai_custom/v1/wenxinworkshop/chat/ernie-4.0-turbo-8k?access_token=%s"
//
```

Figure 10: 修改宏定义

3.1.4 编译烧录

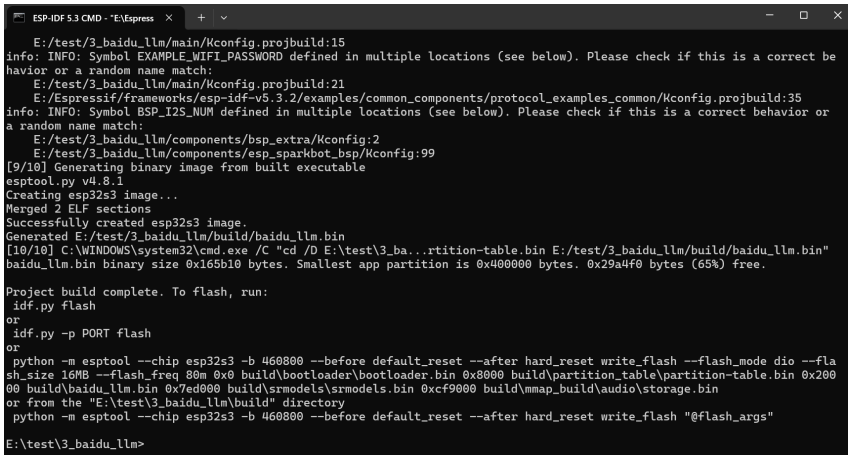
- 打开 ESP-IDF 5.3 CMD
- 输入 **cd 项目文件夹地址**



```
ESP-IDF 5.3 CMD - "E:\Espress
    E:\Espressif\tools\openocd-esp32\v0.12.0-esp32-20241016\openocd-esp32\bin
    E:\Espressif\tools\ninja\1.12.1\
    E:\Espressif\tools\idf-exe\1.0.3\
    E:\Espressif\tools\ccache\4.10.2\ccache-4.10.2-windows-x86_64
    E:\Espressif\tools\dfu-util\0.11\dfu-util-0.11-win64
    E:\Espressif\frameworks\esp-idf-v5.3.2\tools

Checking if Python packages are up to date...
Constraint file: E:\Espressif\espidf.constraints.v5.3.txt
Requirement files:
 - E:\Espressif\frameworks\esp-idf-v5.3.2\tools\requirements\requirements.core.txt
Python being checked: E:\Espressif\python_env\idf5.3_py3.11_env\Scripts\python.exe
Python requirements are satisfied.

Detected installed tools that are not currently used by active ESP-IDF version.
For removing old versions of idf-driver, idf-python-wheels use command 'python.exe E:\Espressif\frameworks\esp-idf-v5.3.
2\tools\idf_tools.py uninstall'
For free up even more space, remove installation packages of those tools. Use option 'python.exe E:\Espressif\frameworks
\esp-idf-v5.3.2\tools\idf_tools.py uninstall --remove-archives'.


Done! You can now compile ESP-IDF projects.
Go to the project directory and run:

  idf.py build


E:\Espressif\frameworks\esp-idf-v5.3.2>cd E:\test\3_baidu_llm

E:\test\3_baidu_llm>
```

Figure 11: CMD

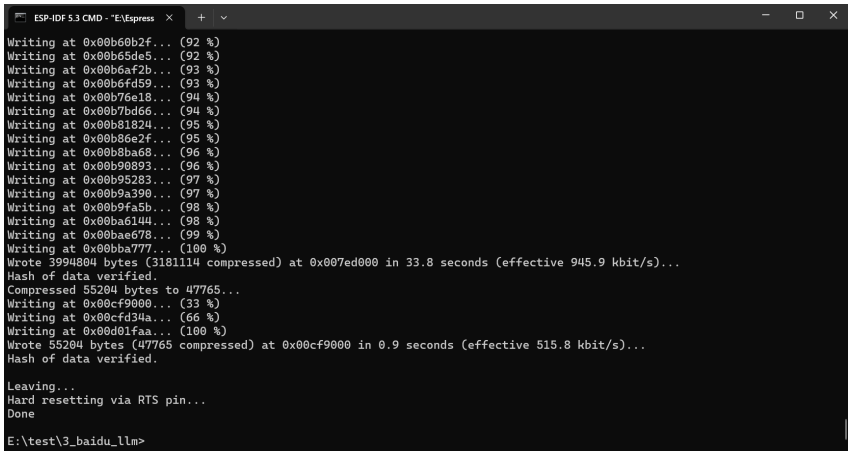
- 输入终端操作指令idf.py build，开始编译



```
ESP-IDF 5.3 CMD - "E:\Espress
    E:/test/3_baidu_llm/main/Kconfig.projbuild:15
info: INFO: Symbol EXAMPLE_WIFI_PASSWORD defined in multiple locations (see below). Please check if this is a correct be
havior or a random name match:
    E:/test/3_baidu_llm/main/Kconfig.projbuild:21
    E:/Espressif/frameworks/esp-idf-v5.3.2/examples/common_components/protocol_examples_common/Kconfig.projbuild:35
info: INFO: Symbol BSP_I2S_NUM defined in multiple locations (see below). Please check if this is a correct behavior or
a random name match:
    E:/test/3_baidu_llm/components/bsp_extra/Kconfig:2
    E:/test/3_baidu_llm/components/esp_sparkbot_bsp/Kconfig:99
[9/10] Generating binary image from built executable
esptool.py v4.8.1
Creating esp32s3 image...
Merged 2 ELF sections
Successfully created esp32s3 image.
Generated E:/test/3_baidu_llm/build/baidu_llm.bin
[10/10] C:\WINDOWS\system32\cmd.exe /C "cd /D E:\test\3_ba...rtition-table.bin E:/test/3_baidu_llm/build/baidu_llm.bin"
baidu_llm.bin binary size 0x165b10 bytes. Smallest app partition is 0x400000 bytes. 0x29a4f0 bytes (65%) free.

Project build complete. To flash, run:
 idf.py flash
or
 idf.py -p PORT flash
or
 python -m esptool --chip esp32s3 -b 460800 --before default_reset --after hard_reset write_flash --flash_mode dio --fla
sh_size 16MB --flash_freq 80m 0x0 build\bootloader\bootloader.bin 0x8000 build\partition_table\partition-table.bin 0x200
00 build\baidu_llm.bin 0x7ed000 build\srmodels\srmodels.bin 0xcf9000 build\mmap_build\audio\storage.bin
or from the "E:\test\3_baidu_llm\build" directory
 python -m esptool --chip esp32s3 -b 460800 --before default_reset --after hard_reset write_flash "@flash_args"

E:\test\3_baidu_llm>
```

Figure 12: 编译完成

- 等待编译完成后，输入idf.py flash，开始烧录



```
ESP-IDF 5.3 CMD - "E:\Espress
Writing at 0x00b60b2f... (92 %)
Writing at 0x00b65de5... (92 %)
Writing at 0x00b6af2b... (93 %)
Writing at 0x00b6fd59... (93 %)
Writing at 0x00b76e18... (94 %)
Writing at 0x00b7bd66... (94 %)
Writing at 0x00b81824... (95 %)
Writing at 0x00b86e2f... (95 %)
Writing at 0x00b8bba68... (96 %)
Writing at 0x00b90893... (96 %)
Writing at 0x00b95283... (97 %)
Writing at 0x00b9a390... (97 %)
Writing at 0x00b9fa5b... (98 %)
Writing at 0x00ba6144... (98 %)
Writing at 0x00bae678... (99 %)
Writing at 0x00bba777... (100 %)
Wrote 3994804 bytes (3181114 compressed) at 0x007ed000 in 33.8 seconds (effective 945.9 kbit/s)...
Hash of data verified.
Compressed 55204 bytes to 47765...
Writing at 0x00cf9000... (33 %)
Writing at 0x00cfd34a... (66 %)
Writing at 0x00d01faa... (100 %)
Wrote 55204 bytes (47765 compressed) at 0x00cf9000 in 0.9 seconds (effective 515.8 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
Done

E:\test\3_baidu_llm>
```

Figure 13: 烧录完成

- 烧录完成后，打开手机热点（使用流量比使用 WIFI 效果更好），会听到继电器吸合的声音，说唤醒词，我使用的是默认唤醒词“Hi，乐鑫。”，可以听到被唤醒的声音，唤醒后即可与乐鑫语音助手进行对话

3.2 屏幕配置

- 为了让语音助手的屏幕更加美观，我重新设计屏幕，并移植到了之前的项目中
- 打开 SquareLine Studio
- 新建文件，选择 Espressif，选择ESP32−S3−EYE

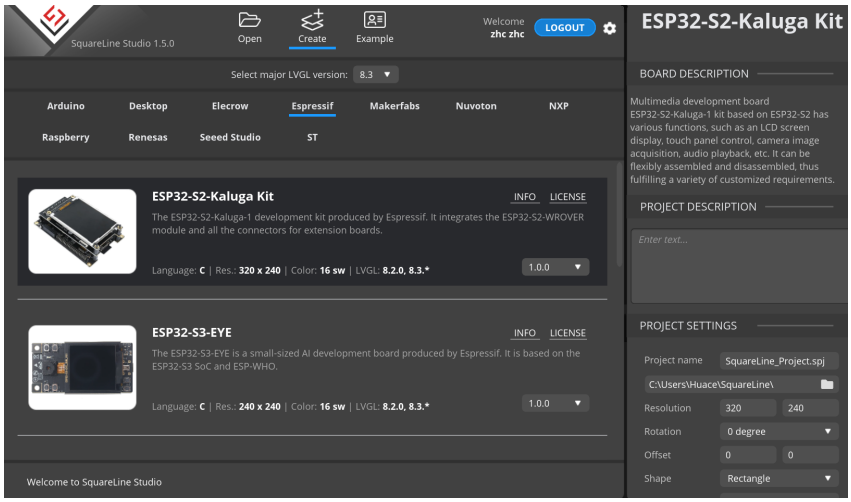


Figure 14: 创建新文件

- 放入元素充实界面内容

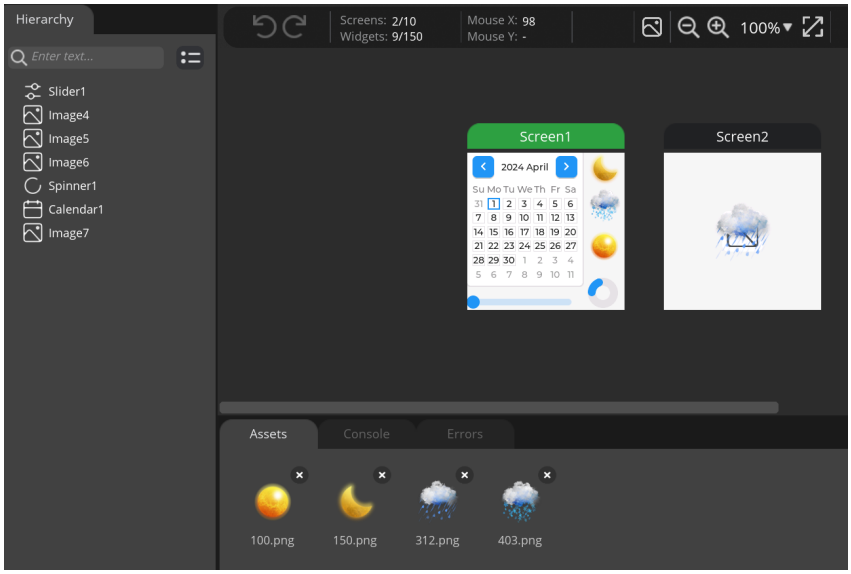


Figure 15: 设计屏幕

- 修改UI Files Export Path，设置为原项目中 UI 文件夹的路径，点击Export，点击Export UI Files
- 删除原项目中的sdkconfig文件，重新编译烧录，可以看到新界面

Figure 16: 屏幕

3.3 演示视频

- 我分别使用了 ERNIE Lite 和 ERNIE 4.0 Turbo 两种大模型，分别询问了两个问题，第一个问题是 “浙江大学的校长是谁?”，第二个问题是 “今天的天气怎么样?”，两种大模型的回答见附件
- 由于 ERNIE Lite 和 ERNIE 4.0 Turbo 均未完全开源，所以我无法再 Hugging Face 上直接对比两种大模型的性能差异。而在百度智能云上对大模型进行评估需要收费，所以我只是通过设置问题做简单的比较
- 演示视频见附件

4 实验思考

- 从演示视频中可以看出，ERNIE 4.0 Turbo 具备联网搜索的能力，很显然 ERNIE Lite 不能联网搜索。ERNIE Lite 的响应时间比 ERNIE 4.0 Turbo 短，但是 ERNIE Lite 的回答错误率明显更高。
- 在实验过程中发现，在修改了代码或者项目中的文件后，如果不删掉原有的 sdkconfig，修改后的内容无法烧录到 ESP 中，那是因为编译过程中新配置并不会覆盖掉 sdcconfig 中原有的配置，所以需要把 sdkconfig 删掉再进行编译并自动生成新的 sdkconfig。
- 在移植新设计的屏幕到原项目时，我为了避免修改原文件，所以生成了一个副本，但是在编译的过程中报错了，即使删掉 sdkconfig 也会报错，那是因为原项目文件夹中存在文件记录有原项目文件夹的路径，这会与新的路径产生冲突。

5 实验心得体会

在这次实验中，我深刻体会到了物联网技术与人工智能结合的强大潜力。通过使用 ESP32 模组和百度千帆大模型，我成功实现了一个能够进行语音交互的智能助手。

实验过程中，我学到了很多新的知识，比如获取 API Key 和 Secret Key 的步骤，以及在修改代码和配置文件时的细节问题。尤其是在移植新设计的屏幕界面时，由于路径冲突导致编译失败，让我意识到项目文件管理的重要性。这些问题的解决过程让我学会了如何更细致地排查错误，也让我对 ESP-IDF 的开发流程有了更深入的理解。

通过对比 ERNIE Lite 和 ERNIE 4.0 Turbo 两种大模型的表现，我直观地感受到了不同模型在响应速度和准确性上的差异。ERNIE 4.0 Turbo 虽然响应稍慢，但其联网搜索能力和回答的准确性明显优于 ERNIE Lite。这让我意识到，在实际应用中，选择合适的大模型需要根据具体需求权衡性能和效率。

这次实验不仅提升了我的技术能力，也让我对物联网和人工智能的结合有了更深的思考。未来，我希望能够进一步优化这个语音助手的功能，比如加入更多本地化的交互设计，或者尝试其他大模型的效果。